

In Java, polymorphism is primarily divided into two main types: **Compile-time Polymorphism** (static binding) and **Runtime Polymorphism** (dynamic binding).

### Compile-Time Polymorphism (Static Polymorphism)

This form of polymorphism is resolved during the compilation phase, where the compiler determines exactly which method to call based on the method's signature (name, number, type, and order of parameters).

- **Method Overloading:** This is the primary way to achieve compile-time polymorphism in Java. It allows a class to have multiple methods with the same name, but with different parameters.
  - **Example:** A Calculator class can have several add methods:

```
java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
    public int add(int a, int b, int c) {
        return a + b + c;
    }
    public double add(double a, double b) {
        return a + b;
    }
}
```

The compiler determines which add method to use based on the number and types of arguments provided in the method call.

- **Operator Overloading (limited):** Java has very limited operator overloading (e.g., the + operator for both numerical addition and string concatenation) and does not allow users to define their own operator overloads.

### 2. Runtime Polymorphism (Dynamic Polymorphism or Dynamic Method Dispatch)

This type of polymorphism is resolved at runtime by the [Java Virtual Machine \(JVM\)](#), not the compiler. It relies on inheritance and the ability of a superclass reference variable to refer to a subclass object.

- **Method Overriding:** This is the primary mechanism for runtime polymorphism. It occurs when a subclass provides a specific implementation for a method that is already defined in its parent class, with the same signature (name, parameters, and return type).

- **Example:** An Animal class with a move() method, overridden by a Dog subclass:

```
java
class Animal {
    public void move() {
        System.out.println("Animals can move");
    }
}
class Dog extends Animal {
    @Override
    public void move() {
        System.out.println("Dogs can run and walk");
    }
}
public class TestDog {
    public static void main(String args[]) {
        Animal b = new Dog(); // Animal reference but Dog object
        b.move(); // Calls the move method in the Dog class at runtime
    }
}
```

The actual method executed depends on the actual type of the object (Dog instance) at runtime, not the reference type (Animal reference). This mechanism is known as [dynamic method dispatch](#).

- Method Overriding is a way (or mechanism or form) to achieve runtime polymorphism
- It is a feature in Java that allows the child class to write its own implementation of a method that is already present in the parent class.
- The JVM determines which method to execute at runtime, based on the object type (not the reference type).
- **Rules of Method Overriding :-**
  - The methods *must have the same name*.
  - The *method must be in different class or in subclass*.
  - All the *method parameters list must be same*:
    - Number of parameters
    - Type of parameters
    - Order of parameters
  - Should follow *IS-A relationship (Inheritance)*.

Advantages of Method Overriding :-

**1. Runtime Polymorphism::**

- Allows Java to decide at runtime which method to execute, enabling flexible and dynamic behavior.

**2. Improves Code Reusability:**

- Reuses method names from the parent class while allowing customized behavior in the child class.

**3. Supports Inheritance:**

- Strengthens the use of inheritance by enabling subclasses to modify or enhance the behavior of parent class methods.

**4. Better Readability and Maintainability:**

- Makes the code more logical and organized when different classes handle behavior in their own way.

#### 5. Encourages Consistency::

- Keeps method names the same across class hierarchies, promoting consistency in design.

#### 6. Useful in Frameworks & Libraries::

- Commonly used in Java frameworks (like Spring, Hibernate) where parent classes define default behavior and subclasses override as needed.

### Important Points

- Some important points to remember about method overriding in Java:
  1. Method overriding requires the subclass method to have the **same name, return type, and parameter list** as the superclass method.
  2. The overriding method must be in a **subclass**, not in the same class.

The access modifier of the overriding method **cannot be more restrictive** than that of the overridden method.

(For example, if the parent method is public, the overriding method cannot be private.)